a separate api.py file for each of your project's apps that participates in the API, with a `Router` in each of those files.

- You create API versioning with Ninja in two steps: (1) use the `version` argument to the `NinjaAPI` object, and then (2) include the version number in your route declaration.
- Ninja provides `Schema` objects to specify how data gets translated into and out of JSON payloads.
- Ninja's `ModelSchema` class is a shortcut for basing a schema on an existing Django Model class.
- Each HTTP method has its own decorator in Ninja. Arguments passed to the decorator can register the ending part of the URL, determine the schema of the view's response, provide a look-up name for the endpoint view, and define the use of an authentication mechanism.
- Ninja endpoints can handle URL-based arguments and query parameters. Both cases get specified using an argument to the endpoint view function, with the URL-based arguments requiring an additional brace-bracket-wrapped token in the URL.
- Python type hints are used within schema definitions to indicate how fields get serialized. The declaration of a schema is similar to Python's `dataclasses` library.
- Ninja schemas look for static class methods with a prefix of `.resolve_` to populate fields. For example, `.resolve_slug()` is called to populate a field named `slug` on the class. The methods get called with an instance of the object getting serialized and should return the field's value.
- Schemas in Ninja can contain fields that reference other schemas, allowing you to nest data. For `Model` classes, a field name corresponding to related objects automatically gets populated. A Python type hint of `list[]` gets used to support the nesting of multiple related objects.
- The `Field` class is a Python type hint that gets used to give fine-grained control over field definitions, including providing default values and field name aliasing.
- Ninja supports a variety of authentication mechanisms, including Django's own authentication system and one based on API keys.
- Authentication can be linked to the `NinjaAPI` object, a `Router` object, or specific endpoint view functions.
- Using an API key that is included in an HTTP header is good practice, as it doesn't require an authentication session on the server, like a Django login does.
- The Django REST Framework is a popular alternative to Django Ninja. It requires more code to implement but also provides more flexibility.
- A GraphQL API can be built into your Django project, using the third-party library Graphene.